Riverton Graduate Research Studio - Program Overview

RGRS-01 | PROGRAM OVERVIEW

SOURCE PAGE 1

The Riverton Graduate Research Studio is a fictional six-week summer program for graduate students from the humanities, social sciences, natural sciences, design, public policy, and computing. The studio is designed for students who want to turn a research question into a small, inspectable prototype. Participants do not need prior programming experience.

The program has three recurring activities. Monday seminars introduce a research or technical idea. Wednesday build labs provide guided implementation time. Friday studios are used for peer feedback, troubleshooting, and project planning. Students may work alone or in teams of two or three.

Every project must begin with a clearly named user, a bounded source collection, and a decision or research task. A prototype is considered complete when another student can identify its inputs, outputs, evidence, and known limitations. The program values small systems that can be inspected over ambitious systems that cannot be explained.

Riverton Graduate Research Studio - Program Overview

RGRS-01 | PROGRAM OVERVIEW

SOURCE PAGE 2

Students choose one of three project paths. The evidence path builds search or question-answering tools over documents. The data path turns messy records into structured information or analysis. The interaction path explores voice, multimodal interfaces, or tool-using agents.

The first two weeks are shared across all paths. Week 1 covers problem definition, source inspection, and basic Python setup. Week 2 covers language models, embeddings, retrieval, and interface design. Specialized workshops begin in Week 3.

The final deliverable is a five-minute demonstration accompanied by a one-page system card. The system card names the intended use, data sources, model or method, human review points, and at least two known failure modes. Projects are demonstrations for learning; they are not approved for clinical, legal, admissions, employment, or other consequential decisions.

Weekly Schedule and Locations

RGRS-02 | SCHEDULE

SOURCE PAGE 1

Monday seminars meet from 10:00 a.m. to noon in Hall B, room 201. The room opens at 9:30 a.m. for coffee and informal questions. Seminars are discussion-based and normally include a short demonstration. Students should bring a notebook; laptops are optional unless the session description says otherwise.

Wednesday build labs meet from 1:00 to 4:00 p.m. in the Innovation Library, room 406. Students should bring a laptop and charger. Loaner laptops are available when reserved by noon on Tuesday. The first thirty minutes are used for a common setup or debugging task; the remaining time is project work with instructors circulating.

Friday project studios meet online from 10:00 to 11:30 a.m. The meeting link is posted in the course portal. Studios begin with a ten-minute check-in, followed by small-group critiques. Students should arrive with one concrete artifact to show: a research question, sample data, retrieval result, interface, or failure trace.

Weekly Schedule and Locations

RGRS-02 | SCHEDULE

SOURCE PAGE 2

Week 1 focuses on choosing a tractable problem and inspecting source material. Monday introduces the program. Wednesday is the environment setup clinic. Friday groups compare project ideas.

Week 2 focuses on retrieval. Monday introduces vectors and similarity. Wednesday is the first RAG build lab. Friday groups inspect retrieved passages and discuss whether the evidence packet is sufficient.

Week 3 covers structured extraction and data quality. Week 4 covers tool use, agents, and permissions. Week 5 is reserved for project integration and user testing. Week 6 includes rehearsal on Wednesday and the public demonstration on Friday afternoon.

If the university closes for severe weather, in-person sessions move online. A change is announced through the portal and email by 8:00 a.m. Students should not rely on a calendar location when an emergency notice has been posted.

Methods Workshops

RGRS-03 | WORKSHOP CATALOG

SOURCE PAGE 1

The Python Foundations clinic is intended for students with little or no programming experience. It covers folders, virtual environments, variables, lists, functions, reading text files, and running a script from the terminal. Examples use research tasks rather than computer-science exercises. The clinic runs during Week 1 Wednesday lab, with an optional repeat session on Thursday morning.

The Document Search workshop introduces chunking, embeddings, cosine similarity, and retrieval. Students build an index over a small set of program documents and create a page that displays the highest-ranked passages. The workshop intentionally postpones answer generation so participants can inspect what retrieval supplied.

The Data-to-Table workshop shows how to define a schema and extract comparable fields from differently written records. It is appropriate for literature reviews, policy documents, archival descriptions, and administrative records.

Methods Workshops

SOURCE PAGE 2

The Interface Prototyping workshop uses Streamlit to turn a Python script into a small browser-based application. Students add a question box, a top-k control, source labels, similarity scores, and expandable passage text. No prior web-development experience is expected.

The Agentic Workflows workshop introduces systems that choose among named tools. Exercises use read-only tools and visible action traces. Students must specify what the agent may do, what requires approval, and what the agent may never access.

The Research Communication workshop helps students explain a prototype to a mixed audience. The recommended demonstration sequence is problem, sample data, system path, one useful result, one failure, and the human decision that remains. Students should avoid presenting a polished answer without showing its evidence.

Workshop materials are posted as notebooks and plain Python scripts. Students may use Codex or another coding assistant, but they remain responsible for running the code, reading errors, and explaining each major transformation.

Computing, Accounts, and Data Storage

RGRS-04 | TECHNICAL GUIDE

SOURCE PAGE 1

Students need Python 3.11 or later, a current browser, and approximately two gigabytes of free disk space. The starter RAG exercise runs on an ordinary laptop. A graphics processor is not required when embeddings are obtained through an API.

The supported setup uses a project-specific virtual environment. Students create the environment, install the supplied requirements file, and keep generated indexes inside the project folder. API keys must be stored in an environment variable or an untracked .env file. Keys must never be pasted into notebooks, screenshots, shared chat messages, or source-control repositories.

Loaner laptops can be reserved through the course portal by noon on the day before a build lab. The computing assistant holds setup office hours immediately before Wednesday lab in Innovation Library 406.

Computing, Accounts, and Data Storage

RGRS-04 | TECHNICAL GUIDE

SOURCE PAGE 2

Course datasets are divided into public, teaching, and restricted categories. Public data may be redistributed when its license permits. Teaching data is fictional or transformed for classroom use and may be shared only with the class unless a file says otherwise. Restricted data includes identifiable research records, confidential administrative records, unpublished interviews, and credentials.

The starter RAG corpus is teaching data. Students may index it locally and may send its text to the approved course API project. It contains no real participant records. Students should use the page-preserving text files for the first implementation and use the combined PDF to inspect the human-facing source.

Generated chunks and embeddings should be written to the output folder. The output folder is excluded from source control because it can be regenerated. Students should preserve document ID, filename, page number, and document type in every chunk so the interface can lead back to the source.

Responsible AI and Data Use Policy

RGRS-05 | POLICY

SOURCE PAGE 1

Students may use approved AI services with public or fictional teaching data. Before sending any other material to an external service, students must confirm that the data owner, consent language, institutional policy, and service terms permit that use. Convenience is not authorization.

Do not upload restricted data to a public AI tool. Restricted data includes identifiable interview transcripts, protected health information, student records, confidential peer review, unpublished partner data, passwords, and API keys. Removing a person's name may not be enough when combinations of details can identify them.

Every project must state whether a model sees the complete source collection, retrieved passages, or only metadata. If a service stores inputs or uses them for another purpose, that behavior must be understood before use.

Responsible AI and Data Use Policy

RGRS-05 | POLICY

SOURCE PAGE 2

AI-assisted code is allowed. Students must review generated dependencies, run the code in the project environment, and be able to explain data movement. A coding assistant should receive a bounded build brief: input files, required output, permitted packages, model choice, storage location, and tests.

For document retrieval projects, the interface must display source identity and passage text before an answer-generation feature is added. The first milestone is evidence discovery, not fluent prose. A system that cannot show what it retrieved is not ready to summarize the collection.

Projects must include a human review point before publishing findings or taking an external action. The reviewer should be able to see the source, model output, uncertainty, and any transformation performed by the system. Evaluation requirements are introduced later in the program; the initial build should remain small enough that students can inspect every retrieved passage manually.

Mentoring, Office Hours, and Learning Support

RGRS-06 | STUDENT SUPPORT GUIDE

SOURCE PAGE 1

Each project team is assigned a primary mentor by the end of Week 1. Mentors help narrow the research question, identify a suitable source collection, and choose a feasible technical path. Mentors do not write the project or approve the use of restricted data.

General office hours are held Monday from 1:00 to 3:00 p.m. in Hall B, room 214, and Thursday from 10:00 a.m. to noon online. Technical debugging hours are held Wednesday from noon to 1:00 p.m. in Innovation Library 406. Students may attend without an appointment, but teams requesting a data review should upload a short source description in advance.

Students new to Python are encouraged to attend the Foundations clinic and bring one error or confusing command to office hours. Beginners are not expected to build the project alone from a blank file.

Mentoring, Office Hours, and Learning Support

RGRS-06 | STUDENT SUPPORT GUIDE

SOURCE PAGE 2

Peer learning groups meet during Friday studios. The groups are deliberately mixed by discipline and technical experience. A useful peer critique identifies what the system takes in, what it returns, what evidence is visible, and what claim would be unsafe.

The writing consultant offers twenty-minute appointments in Weeks 4 through 6 for system cards, project descriptions, and demonstrations. The consultant does not verify technical performance but can help distinguish evidence, interpretation, and recommendation.

Students who fall behind should contact the primary mentor before expanding their project. The recommended recovery plan is to reduce the source collection, keep retrieval visible, and postpone answer generation or agentic features. A smaller working system is preferred to an unfinished system with more components.

Team Project Guidelines

RGRS-07 | PROJECT REQUIREMENTS

SOURCE PAGE 1

Teams contain one to three students. By the end of Week 1, every team submits a project brief naming the user, problem, source collection, proposed output, and one consequential error. The brief must also state whether the project uses public, teaching, or restricted data.

The Week 2 checkpoint is a retrieval or transformation trace. Document projects submit sample chunks and at least three query results. Data projects submit a schema and two extracted records. Interaction projects submit a transcript or action trace. A polished interface is not required at this checkpoint.

The Week 4 checkpoint is a working interface that exposes sources or tool actions. Teams should not hide the intermediate evidence behind a single answer box. Each team receives one structured peer review during Friday studio.

Team Project Guidelines

RGRS-07 | PROJECT REQUIREMENTS

SOURCE PAGE 2

Project repositories must include a README with setup instructions, a data manifest, a requirements file, and a short statement explaining which outputs can be regenerated. Secrets and restricted data must not be committed.

Students may divide roles, but everyone should understand the complete path from input to output. During the final demonstration, the instructor may ask any team member to explain chunking, model selection, retrieval, or a failure case.

The scope-change deadline is the end of Week 3. After that point, teams may simplify a project without approval but should not add a new sensitive dataset or consequential use. The primary mentor can approve changes to the research question when the existing collection cannot answer it.

Accessibility, Participation, and Wellbeing

RGRS-08 | PARTICIPATION GUIDE

SOURCE PAGE 1

Course materials are provided in accessible HTML or tagged PDF when possible. Slides use high-contrast colors and include text alternatives for meaningful images. Students may request materials in another format through the program coordinator.

Participation can take several forms: speaking during discussion, contributing through the shared notes, presenting a small artifact, testing another team's interface, or submitting a written reflection. Camera use is optional in online studios. Students should not record a session or another participant without explicit permission.

Students who use screen readers or keyboard navigation should tell the interface workshop instructor which tools they use so example applications can be tested appropriately.

Accessibility, Participation, and Wellbeing

RGRS-08 | PARTICIPATION GUIDE

SOURCE PAGE 2

The program includes two short breaks during the three-hour Wednesday lab. Students may step out when needed and may work in the adjacent quiet room. Food is not allowed near loaner equipment, but covered drinks are permitted.

Students can request deadline flexibility for disability, illness, caregiving, religious observance, or significant access barriers. Requests should be made to the program coordinator; students are not required to disclose medical details to classmates or project mentors.

Project teams should plan work so that a single member is not the only person able to run the system. Shared setup notes, paired debugging, and small reproducible scripts reduce both technical risk and uneven workload.

Fieldwork, Travel, and Reimbursements

RGRS-09 | ADMINISTRATIVE POLICY

SOURCE PAGE 1

Small project expenses may be reimbursed when they are necessary for an approved studio project. Examples include local transit for field observation, participant refreshments, printing, and a short-term software license that is not already supplied by the university.

Students must obtain written mentor approval before spending money. The approval should name the project, expense category, expected amount, and purpose. Purchases made before approval may be ineligible. The standard project limit is 250 dollars unless the program director approves an exception.

Original itemized receipts and proof of payment are required. Reimbursement requests must be submitted through the course portal within ten calendar days of the expense.

Fieldwork, Travel, and Reimbursements

RGRS-09 | ADMINISTRATIVE POLICY

SOURCE PAGE 2

Local fieldwork should be planned with attention to safety, accessibility, privacy, and participant burden. Students should tell a teammate or mentor where the activity will occur and when they expect to return. Field observation does not automatically authorize photography, recording, or collection of personal information.

Overnight travel is outside the standard studio budget and requires advance approval from the program director and the university travel office. International travel is not supported through the studio project fund.

Questions about eligible expenses should be sent to the program administrator before purchase. The reimbursement policy does not cover personal equipment, routine meals, fines, or expenses unrelated to an approved project.

Demonstration Day and Frequently Asked Questions

RGRS-10 | EVENT GUIDE AND FAQ

SOURCE PAGE 1

Demonstration Day takes place Friday of Week 6 from 1:00 to 4:00 p.m. in the Innovation Library event space. Each team receives five minutes to demonstrate and three minutes for questions. A rehearsal is held during Wednesday lab.

The recommended presentation sequence is: define the problem, show the source collection, trace one input through the system, demonstrate one useful result, show one failure or limitation, and name the human decision that remains. Live systems should have a recorded or static backup.

Final materials are due at noon on Thursday of Week 6. Teams submit the repository link, one-page system card, three representative screenshots, and the demonstration title. The system card must list data sources, model or method, known limitations, and the person responsible for final review.

Demonstration Day and Frequently Asked Questions

RGRS-10 | EVENT GUIDE AND FAQ

SOURCE PAGE 2

Frequently asked questions:

Can a student participate without programming experience? Yes. Attend Python Foundations, use the supplied starter project, and bring setup problems to technical office hours.

Does the program provide housing? The studio documents do not describe housing or housing support. Students should consult the university's general student-services information.

Can a team use its own research data? Possibly, but only after confirming authorization and data classification. The fictional starter corpus should be used for the first build.

Can a team use an AI coding assistant? Yes. The team must review the generated code, protect credentials and restricted data, and be able to explain the implementation.

What if the interface is not finished? Demonstrate the working pipeline in the terminal, show retrieved evidence, and explain the next implementation step.